LAPORAN TUGAS

STURKTUR DATA DAN ALGORITMA
PENJELASAN TAMBAHAN TUGAS 3 QUEUE

Dosen Pengampu:

Prof. Dr. Taufik Fuadi Abidin, M.Tech

Disusun oleh:

Muhammad Aqil Mubarak (250810701100003)

JURUSAN INFORMATIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS SYIAH KUALA

BANDA ACEH

2026

PROBLEM TUGAS 3

Problem 1

Anda diminta untuk membuat program untuk menyelesaikan masalah Simulasi Antrian Printer pada sebuah laboratorium komputer. Pada lab tersebut terdapat satu printer yang digunakan bersama oleh mahasiswa. Setiap dokumen yang dikirim ke printer akan masuk ke dalam sebuah antrian dan diproses sesuai urutan kedatangannya. Secara rinci, anda diminta untuk melakukan hal-hal sebagai berikut:

1. Membuat program simulasi antrian printer menggunakan queue yang diimplementasikan menggunakan array dan linked list.

2. Program harus memiliki menu untuk:

    a. Menambah dokumen ke antrian cetak

    b. Mencetak dokumen berikutnya

    c. Menampilkan seluruh antrian dokumen

    d. Menampilkan dokumen yang akan dicetak atau diproses berikutnya

3. Tampilkan pesan yang menjelaskan ketika antrian kosong.

Problem 2

Anda diminta untuk membuat program untuk menyelesaikan masalah Simulasi Antrian Pelayanan Bank. Bank memiliki dua jalur layanan yang dapat digunakan oleh para nasabah. Setiap nasabah yang datang diminta untuk mengambil nomor antrian dan dilayani berdasarkan urutan kedatangannya pada kedua jalur layanan. Secara rinci, anda diminta untuk melakukan hal-hal sebagai berikut:

1. Buat program simulasi antrian nasabah bank menggunakan queue yang diimplementasikan menggunakan array dan linked list.

2. Program harus menyediakan menu:

    a. Menambah nasabah ke jalur antrian dengan antrian terpendek

    b. Melayani nasabah berikutnya

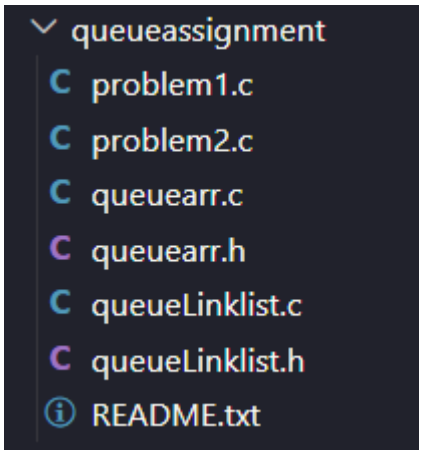
    c. Menampilkan seluruh antrian nasabah pada setiap jalur

    d. Menampilkan jumlah nasabah yang sedang menunggu pada setiap jalur

    e. Menampilkan ID atau nomor nasabah yang akan diproses

PENJELASAN TAMBAHAN

Dalam Pengerjaan solusi 2 problem ini, saya menggunakan pendekatan modularitas, dimana program dipecah menjadi beberapa file terpisah yaitu file header dan file implementasi. Cara ini memisahkan logika dasar struktur data dengan logika utama penyelesaian pada problem 1 dan problem 2, sehingga kode menjadi rapi dan mudah di kelola.



Gambar 1. Direktori File

Problem 1 dapat di compile dengan command:

'gcc -Wall -pedantic -g -o problem1 problem1.c queuearr.c queueLinklist.c'

Problem 2 dapat di compile dengan command:

'gcc -Wall -pedantic -g -o problem2 problem2.c queuearr.c queueLinklist.c'

Problem 1

Problem 1, yang pertama ini adalah antrian printer, dimana konsep FIFO (First In First Out) di implementasikan dengan dua struktur data yang berbeda, yaitu Array dan Linked List. Dalam implementasi array saya mendefinisikan struct 'QueueArr' dengan alokasi memori statis dan dengan batas maksimal 100 dokumen '#define MAX 100'.

```c
#define MAX 100

// Struct Queue menggunakan implementasi Array
typedef struct queue {
    int data[MAX];
    int front;
    int rear;
    int count;
} QueueArr;
```

Gambar 2. Struct Queue Array

Kemudian dalam menghindari pemborosan ruang pada array karena berganti antrian, pada fungsi ‘enqueueArr’ dan ‘dequeueArr’, penunjuk ‘rear’ dan ‘front’ menggunakan operator modulo ‘% MAX’. Dimana logika ini memastikan saat penunjuk mencapai indeks ke-99, penunjuk ini akan otomatis kembali ke indeks 0 jika masih ada ruang kosong. Untuk kode unik dokumen saya membuat variabel ‘idDocs’ untuk menyimpan dan melacak ID dokumen dimana variabel ini akan terus di increament setiap kali ada dokumen baru.

```c
// Fungsi untuk memasukkan data baru ke Queue
void enqueueArr(QueueArr *queue, int data) {
    if (isFullArr(queue)) {
        printf("Queue penuh\n");
        return;
    }
    queue->rear = (queue->rear + 1) % MAX;
    queue->data[queue->rear] = data;
    queue->count++;
}
```

Gambar 3. Fungsi enqueueArr

Kemudian dalam implementasi Linked List, berbeda dengan array yang ukuran queue nya statis, pada implementasi Linked List ukuran queue nya menjadi dinamis. Dimana setiap kali fungsi ‘enqueueLL’ di panggil, program akan meminta alokasi memori baru ke sistem menggunakan fungsi ‘malloc()’ untuk membuat ‘QueueNode’ baru yang disambungkan ke ‘rear’ queue. Kemudian Saat dokumen dicetak menggunakan ‘dequeueLL’, node yang berada di ‘front’ akan diambil datanya dan langsung dihapus dari memori memakai fungsi ‘free()’.

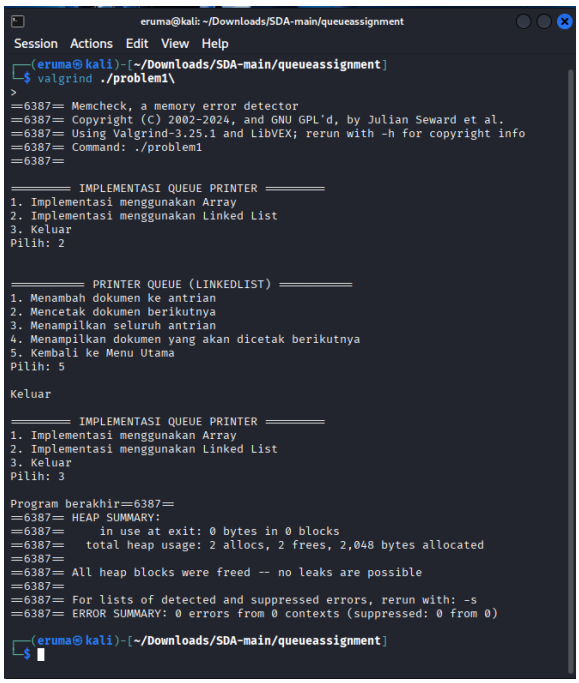
```c
void enqueueLL (QueueLL *queue, int data) {
    QueueNodePtr newNode = (QueueNodePtr)malloc(sizeof(QueueNode));

    if (newNode == NULL) {
        printf ("Gagal melakukan alokasi memori\n");
        return;
    }
    newNode->data = data;
    newNode->next = NULL;

    if (isEmptyLL(queue)) {
        queue->front = newNode;
        queue->rear = newNode;
    } else {
        queue->rear->next = newNode;
        queue->rear = newNode;
    }
    queue->size++;
}
```

Gambar 4. Fungsi enqueueLL

```c
int dequeueLL (QueueLL *queue) {
    if (isEmptyLL(queue)) {
        return -1;
    }
    QueueNodePtr temp = queue->front;
    int dequeueData = temp->data;

    queue->front = queue->front->next;

    if (queue->front == NULL) {
        queue->rear = NULL;
    }

    free(temp);
    queue->size--;

    return dequeueData;
}
```

Gambar 5. Fungsi dequeueLL

Kemudian proses pengecekan memori, memastikan tidak ada sisa memori yang bocor saat program selesai di jalankan. Pada screenshot dibawah program mencetak status “All heap blocks were freed – no leaks are posibble”, yang mengonfirmasi bahwa alokasi memori dinamis (heap) telah sukses.



Gambar 6. Memory Check Problem 1

Output Problem 1:

```
========= IMPLEMENTASI QUEUE PRINTER =========
1. Implementasi menggunakan Array
2. Implementasi menggunakan Linked List
3. Keluar
Pilih: 1


============= PRINTER QUEUE (ARRAY) =============
1. Menambah dokumen ke antrian
2. Mencetak dokumen berikutnya
3. Menampilkan seluruh antrian
4. Menampilkan dokumen yang akan dicetak berikutnya
5. Kembali ke Menu Utama
Pilih:
```

Gambar 7. Output Printer Queue (Array)

```
========= IMPLEMENTASI QUEUE PRINTER =========
1. Implementasi menggunakan Array
2. Implementasi menggunakan Linked List
3. Keluar
Pilih: 2


=========== PRINTER QUEUE (LINKEDLIST) ===========
1. Menambah dokumen ke antrian
2. Mencetak dokumen berikutnya
3. Menampilkan seluruh antrian
4. Menampilkan dokumen yang akan dicetak berikutnya
5. Kembali ke Menu Utama
Pilih:
```

Gambar 8. Output Printer Queue (LinkedList)

Problem 2

Problem 2, yang kedua ini sedikit berbeda dengan yang pertama, kalau pertama hanya ada 1 antrian/queue, yang kedua ini ada dua antrian/queue dimana dua antrian ini berjalan seacara bersamaan dan mengelola nasabah baru secara otomatis ke jalur antrian yang lebih sepi sehingga antrian dapat selesai lebih cepat.

Dalam implementasi array, saya menginisialisasi dua antrian dengan variabel ‘jalur1’ dan ‘jalur2’. Saat nasabah ingin mengambil antrian baru, program akan mengevaluasi kondisi antrian dengan membandingkan nilai atribut dari ‘.count’ dari kedua jalur antrian. Dimana jika ‘jalur1.count <= jalur2.count’, maka nasabah akan diarahkan masuk ke jalur 1 menggunakan fungsi ‘enqueueArr()’.

```c
case 1:
    if (jalur1.count <= jalur2.count) {
        enqueueArr(&jalur1, idNasabah);
        printf("Nasabah ID %d masuk ke Jalur 1\n", idNasabah);
    } else {
        enqueueArr(&jalur2, idNasabah);
        printf("Nasabah ID %d masuk ke Jalur 2\n", idNasabah);
    }
    idNasabah++;
    break;
```

Gambar 9. Case 1 pemilihan jalur array

Kemudian pada implementasi Linked List pemilihan jalur diterapkan sama dengan parameter pembanding adalah atribut '.count' dimana 'jalur1.count <= jalur2.count'. Sesuai dengan definisi data pada struct QueueLL. Kemudian setiap nasabah baru akan dialokasikan memorinya menggunakan 'malloc()' dan disambungkan ke ujung antrian 'rear' pada jalur yang memenuhi syarat pengeekan.

```c
case 1:
    if (jalur1.count <= jalur2.count) {
        enqueueLL(&jalur1, idNasabah);
        printf("Nasabah ID %d masuk ke Jalur 1\n", idNasabah);
    } else {
        enqueueLL(&jalur2, idNasabah);
        printf("Nasabah ID %d masuk ke Jalur 2\n", idNasabah);
    }
    idNasabah++;
    break;
```

Gambar 10. Case 1 pemilihan jalur linked list

Kemudian karena pada implementasi Linked List pada problem 2 ini menjalankan dua antrian dinamis sekaligus, kita perlu membersihkan memori di akhir programnya juga, jadi pada program ini saat user memilih opsi ke-6 yaitu keluar, maka opsi tersebut menjalankan fungsi 'freeQueueLL()' yang dipanggil secara berurutan untuk referensi memori &jalur1 dan &jalur2. Hal ini akan membuat seluruh node dari nasabah di kedua jalur yang belum sempat dilayani (dequeue) ikut dihapus dari heap memory.

```c
case 6:
    printf("Keluar\n");
    freeQueueLL(&jalur1);
    freeQueueLL(&jalur2);
    break;
```

Gambar 11. Case 6 Linked List

Kemudian proses pengecekan memori, memastikan tidak ada sisa memori yang bocor saat program selesai di jalankan. Pada screenshot dibawah program mencetak status "All heap blocks were freed – no leaks are posibble", yang mengonfirmasi bahwa alokasi memori dinamis (heap) telah sukses.

```
eruma@kali: ~/Downloads/SDA-main/queueassignment
Session  Actions  Edit  View  Help
└─$ valgrind ./problem2
==10318== Memcheck, a memory error detector
==10318== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==10318== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==10318== Command: ./problem2
==10318==

======== IMPLEMENTASI QUEUE BANK ========
1. Implementasi menggunakan Array
2. Implementasi menggunakan Linked List
3. Keluar
Pilih: 2


================ ANTRIAN BANK (LINKED LIST) ================
1. Menambah nasabah ke jalur antrian terpendek
2. Melayani nasabah berikutnya
3. Menampilkan seluruh antrian nasabah
4. Menampilkan jumlah nasabah yang menunggu
5. Menampilkan ID Nasabah yang akan diproses
6. Kembali ke Menu Utama
Pilih: 6

Keluar

======== IMPLEMENTASI QUEUE BANK ========
1. Implementasi menggunakan Array
2. Implementasi menggunakan Linked List
3. Keluar
Pilih: 3

Program berakhir
==10318==
==10318== HEAP SUMMARY:
==10318==     in use at exit: 0 bytes in 0 blocks
==10318==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==10318==
==10318== All heap blocks were freed -- no leaks are possible
==10318==
==10318== For lists of detected and suppressed errors, rerun with: -s
==10318== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

┌──(eruma㉿kali)-[~/Downloads/SDA-main/queueassignment]
└─$
```

Gambar 12. Memory Check Problem 2

Output Problem 2:

```
> gcc -Wall -pedantic -g -o problem2 problem2.c queuearr.c queueLinklist.c
> .\problem2

========= IMPLEMENTASI QUEUE BANK =========
1. Implementasi menggunakan Array
2. Implementasi menggunakan Linked List
3. Keluar
Pilih: 1


================ ANTRIAN BANK (ARRAY) ================
1. Menambah nasabah ke jalur antrian terpendek
2. Melayani nasabah berikutnya
3. Menampilkan seluruh antrian nasabah
4. Menampilkan jumlah nasabah yang menunggu
5. Menampilkan ID Nasabah yang akan diproses
6. Kembali ke Menu Utama
Pilih:
```

Gambar 14. Output Antrian Bank (Array)

```
========= IMPLEMENTASI QUEUE BANK =========
1. Implementasi menggunakan Array
2. Implementasi menggunakan Linked List
3. Keluar
Pilih: 2


================ ANTRIAN BANK (LINKED LIST) ================
1. Menambah nasabah ke jalur antrian terpendek
2. Melayani nasabah berikutnya
3. Menampilkan seluruh antrian nasabah
4. Menampilkan jumlah nasabah yang menunggu
5. Menampilkan ID Nasabah yang akan diproses
6. Kembali ke Menu Utama
Pilih:
```

Gambar 15. Output Antrian Bank (LinkedList)